PHISHWARE

Analisis Tecnico Integral del Sistema

Backend + Frontend web + App Android + Extension + Infraestructura Docker

Auditoria de ingenieria con verificacion en vivo - Noviembre 2026

PHISHWARE - Análisis Técnico Integral del Sistema

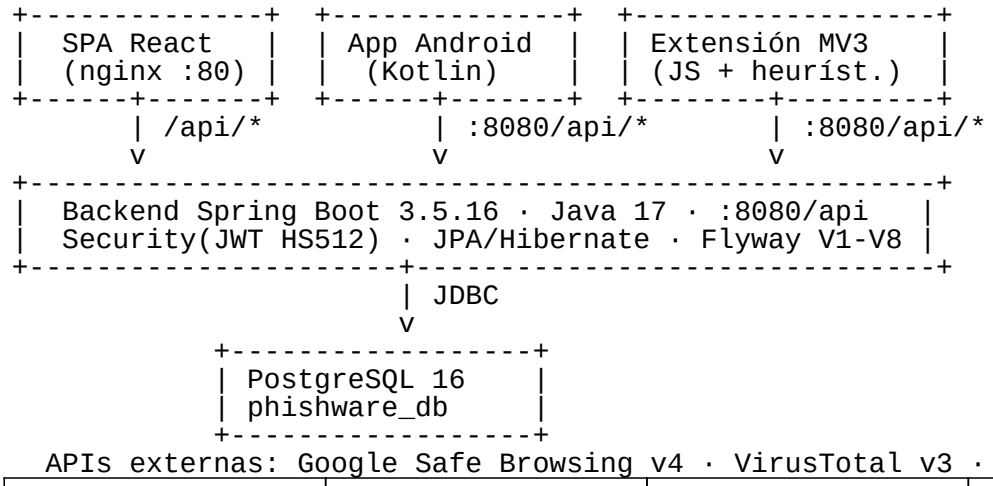
Versión del documento: 1.0 - 9 de noviembre de 2026 **Autor:** Auditoría de ingeniería (código + despliegue verificado en vivo) **Alcance:** backend, frontend web, app Android, extensión de navegador, infraestructura Docker

1. Resumen ejecutivo

PHISHWARE es una plataforma anti-phishing multi-cliente con un backend central en Spring Boot que expone una API REST (/api), consumida por tres clientes independientes: una SPA web en React, una app Android nativa en Kotlin y una extensión de navegador Manifest V3. El valor diferencial real del sistema es su motor de riesgo combinado: seis fuentes independientes (listas negras comerciales, heurística léxica, scraping de contenido, antigüedad del dominio y reportes comunitarios) se fusionan con una compuerta ruidosa (noisy-OR), riesgo = 1 - Pi(1 - p_i), de modo que una señal fuerte nunca se diluye y varias señales débiles se refuerzan. A esto se suma un módulo educativo con microlecciones mapeadas a principios de persuasión (urgencia, autoridad, escasez) y gamificación (puntos, niveles, rachas, insignias), más reportes comunitarios confirmados.

El sistema está desplegado con Docker Compose (Postgres 16 + backend + frontend nginx) y fue verificado extremo a extremo: registro, login, análisis de URL, educación, gamificación, notificaciones push en Android y segundo plano.

2. Arquitectura del sistema



Componente	Tecnología y versión	Puerto / ruta	Estado verificado
backend	Spring Boot 3.5.16, Java 17, Tomcat 10.1	8080, context-path /api	OK (Flyway V8, 11 repos)
postgres	postgres:16-alpine	5432, vol. persistente	OK (healthy)

Componente	Tecnología y versión	Puerto / ruta	Estado verificado
frontend	React 18.3 + TS 5.4 + Vite 7 + Tailwind 3.4, nginx:alpine	3000->80, proxy /api/	OK
android	Kotlin, compileSdk/targetSdk 34, minSdk 26, v1.0.0 (flavors full/play)	127.0.0.1:8080 vía adb reverse	OK
extension	Manifest V3 (popup/options/background SW/content/heuristics)	localhost:8080, 10.0.2.2	Presente, pendiente E2E

Por qué esta arquitectura

- **Un solo backend, tres clientes.** Toda la inteligencia (riesgo, educación, comunidad) vive en el servidor; los clientes son delgados. Cambiar el algoritmo no exige actualizar apps instaladas.
- **server.servlet.context-path=/api.** Aísla la API del contenido estático y permite que nginx sirva web y API bajo el mismo origen (sin CORS en producción).
- **Postgres + Flyway, no H2.** Persistencia real con migraciones versionadas (V1-V8); el historial valida checksums para impedir drift de esquema.

---

3. Backend - Spring Boot

3.1 Stack (pom.xml verificado)

Spring Boot 3.5.16 (parent), Java 17, spring-boot-starter-web, data-jpa, security, validation, actuator, postgresql (driver), flyway-core + flyway-database-postgresql, jjwt 0.12.5 (api/impl/jackson), mapstruct 1.5.5, lombok 1.18.38, springdoc-openapi 2.5.0 (Swagger en /swagger-ui.html). Build multi-stage: maven:3.9-eclipse-temurin-17 -> eclipse-temurin:17-jre, usuario no-root phishware.

3.2 Seguridad

- **JWT HS512** (JwtTokenProvider): access 24 h + refresh 7 días. Filtro JwtAuthenticationFilter (OncePerRequestFilter) anterior a UsernamePasswordAuthenticationFilter. Entry point propio devuelve 401 JSON {"status":401,...,"message":"Token inválido o sesión expirada"}.
- **BCrypt cost 12** para contraseñas; DaoAuthenticationProvider con hideUserNotFoundExceptions=true (no revela si el usuario existe).
- **Endurecimiento aplicado en esta auditoría:** el filtro JWT captura cualquier excepción (token válido pero usuario inexistente, p. ej. DB recreada) y sigue la cadena en vez de romper endpoints

públicos (/auth/**).

- **Rate limiting** por IP (RateLimitingConfig, 120 req/min) en /auth/login, /auth/register, /analysis, /public/url, /security/password-check; responde 429 con cabeceras X-RateLimit.
- **Cabeceras OWASP** (SecurityHeadersConfig): nosniff, DENY frame-options, HSTS, CSP estricta, referrer-policy, permissions-policy, no-store en /api/.
- **Validación de entrada OWASP A03/A10** (InputSanitizerService, OwaspComplianceValidator): esquemas http/https, longitud 2048, bloqueo de XSS/SQLi/command injection, anti-SSRF (RFC 1918, metadata cloud, resolución DNS con SafeNetwork). **Caso real corregido:** el regex on\w+\s*= marcaba como XSS el parámetro legítimo sessionExpired= (contiene onExpired=); se acotó con lookbehind de no-palabra para no romper URLs de campus virtuales.

3.3 Persistencia

JPA/Hibernate (ddl-auto: validate - el esquema lo gobierna Flyway, nunca Hibernate), 11 repositorios. Migraciones: V1 esquema, V2 datos iniciales (roles + admin seed), V3 contenido OWASP/NIST, V4 reportes comunitarios, V5 quiz por módulo, V6 fix quiz JSON, V7 microlecciones de persuasión, **V8 fix del hash BCrypt del admin** (el seed V2 traía un hash truncado de 52 caracteres que hacía imposible el login del admin; V8 lo corrige sin romper el checksum de V2).

3.4 Motor de análisis (corazón del producto)

UrlAnalysisService.analyzeUrl: sanitiza -> anti-SSRF -> 4 fuentes externas en paralelo (pool de 16 hilos, timeouts 8-20 s con fallback degradado) + heurística local + veredicto comunitario -> noisy-OR -> persiste análisis, amenazas, alerta (NIST RS) y puntos de gamificación.

Fuente	Confianza	Aporte
Google Safe Browsing	lista negra	0.95 si peligroso
VirusTotal (consenso)	consenso	0.5-0.95 / 0.08·sospechosos
Heurística léxica NIST	0.75	score/100 × 0.75 (siempre disponible)
Contenido scraping (Jsoup)	0.85	score/100 × 0.85
Antigüedad dominio (RDAP)	0.70	señales de dominio joven
Comunidad CONFIRMED	0.90	veredicto directo a phishing

riesgo = 1 - Pi(1 - p_i); umbrales: 70 phishing, 50 DANGEROUS, 20 SUSPICIOUS, resto SAFE. Cada amenaza guarda severidad, fuente y principio de persuasión (mapeo IndicatorEducationCatalog) para enlazar con microlecciones.

3.5 Módulos y endpoints principales

Auth (/auth/login|register|refresh), análisis (/analysis, historial paginado), dashboard agregador, educación + quiz por módulo (/education, /quiz), gamificación (puntos/rachas/insignias), reportes comunitarios con flujo de moderación, alertas (no leídas, marcar leída), admin (/admin/** rol ADMIN), utilidades (/public/url, /security/password-check). Documentación OpenAPI con springdoc; salud en /actuator/health.

4. APIs y servicios externos

Integración	Versión / costo	Uso	Observación
Google Safe Browsing	v4, API key server-side	listas negras de phishing/malware	threatMatches en paralelo
VirusTotal	v3, API key server-side	consenso de ~70 motores	Malicious 50%+motores
RDAP (rdap.org)	sin key	fecha de creación del dominio	dominios jóvenes = riesgo
Scraping propio (Jsoup)	timeout 8 s, 1.5 MB, 5 redirects	indicadores en HTML visible	con SafePageFetcher anti-SSRF

Las keys viven en el .env del servidor, nunca en clientes. Todo fallo externo degrada a "fuente no disponible" sin tumbar el análisis (timeouts + fallbacks).

5. Frontend web - React

React 18.3 + TypeScript 5.4 + Vite 7 + Tailwind 3.4 + axios 1.7 + react-router + react-query + react-hook-form + react-hot-toast; i18n propio es/en (Spanish-first); dark mode por clase (darkMode:'class' + ThemeContext con modo system). Design system documentado (DESIGN.md + .impeccable/design.json, North Star "The Campus Safety Console"): vino #790D2B, raspberries, semáforos verde/ámbar/rojo solo para riesgo, Inter + JetBrains Mono para URLs, radios 12/16 px, sombras contenidas.

Páginas: Landing, Login, Register, Dashboard, UrlAnalysis (tabs URL/QR), History, Education + Article, Quiz (ModuleQuiz), Alerts, Reports, Profile, Settings, Admin. api.ts: baseURL /api, inyecta Bearer, ante 401 en /auth/ limpia sesión obsoleta (fix de esta auditoría) y ante 401/403 global redirige a login. Servido por nginx que proxifica /api/ al backend con resolver Docker + $request_uri completo (fix de esta auditoría: con variables nginx no reescribe el URI; la forma anterior duplicaba el prefijo a /api/api/...).

6. App Android - Kotlin nativa

Kotlin, MVVM + Hilt (DI), Retrofit2 + OkHttp (logging BODY en debug) + Gson, DataStore Preferences (sesión + toggles Guard), WorkManager (SyncWorker cada 15 min), Navigation Component (NavHost + BottomNavigationView + Toolbar con Up), Material3. minSdk 26, targetSdk/compileSdk 34, v1.0.0, flavors full (con SMS) y play (sin SMS). BASE_URL debug http://127.0.0.1:8080/api/ (por adb reverse en físico; 10.0.2.2 en emulador).

Segundo plano (Ley 1581: minimización - nunca sale el cuerpo del mensaje): LinkGuardService (foreground + notificación persistente) con reintento Boot; PhishNotificationListener (NotificationListenerService) vigila WhatsApp, Gmail, Outlook, Telegram y Messenger: extrae solo URLs del texto visible y las analiza; SmsReceiver hace lo propio con SMS entrantes (con fallback local MessageTextAnalyzer si no hay backend); ShareReceiverActivity recibe texto compartido; SyncWorker refresca no-leídos e insignia. GuardPreferences puerta todo tras isActive() = consent && guard.

Hallazgos de esta auditoría (corregidos): el diálogo de consentimiento no guardaba nada (botón sin acción -> todo el segundo plano muerto); el botón de prueba de notificación no pedía POST_NOTIFICATIONS; se añadió toolbar con flecha atrás (Up solo fuera de destinos raíz); se eliminaron todos los emojis de la UI (vectores + puntos ` coloreados víaRiskUi`), y la píldora de búsqueda ahora abre el analizador.

7. Extensión de navegador - Manifest V3

manifest.json MV3: background.js (service worker, type module), content.js, heuristics.js (análisis léxico local inmediato), popup.html/js, options.html/js, iconos y _locales (i18n). Permisos mínimos: activeTab, storage, notifications, alarms, declarativeNetRequest; hosts http://localhost:8080/* y http://10.0.2.2:8080/* (desarrollo). Filosofía coherente con Android: veredicto local instantáneo + confirmación del backend.

8. Infraestructura Docker

docker-compose.yml: postgres:16-alpine (volumen phishware-postgres-data, healthcheck pg_isready), backend (build multi-stage, depends_on postgres healthy, SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/..., healthcheck TCP 8080), frontend (build Node 22 + nginx, depends_on backend healthy). Decisiones de esta auditoría: URL de BD por variable de entorno (antes localhost hardcodeado rompía Flyway dentro del contenedor), healthchecks para ordenar el arranque (antes nginx moría con host not found in upstream), y proxy nginx resiliente a reinicios del backend.

9. Modelo de datos (entidades JPA)

User (BCrypt, roles vía Role/user_roles, puntos/nivel/racha), UrlAnalysis (url, dominio, protocolo, riskLevel/Score, isPhishing, source, rawResponse JSON, tiempos), Threat (tipo, descripción, severidad, fuente, principio de persuasión), Alert, CommunityReport (flujo de moderación), Badge/UserBadge, EducationalContent, Quiz/QuizQuestion/QuizResult, AuditLog. Relaciones análisis->amenazas y usuario->análisis indexadas para el dashboard e historial paginado.

10. Decisiones de ingeniería y su porqué

Decisión	Porqué
Noisy-OR en vez de promedio ponderado	**una señal fuerte nunca se diluye; fuentes caídas simplemente no participan**
Timeouts + fallbacks por fuente	**GSB/VT/RDAP lentos o caídos no bloquean el veredicto**
ddl-auto: validate + Flyway	**el esquema es código versionado y auditable; Hibernate no toca la BD**
V8 en vez de editar V2	**Flyway valida checksums; editar migraciones aplicadas rompe el arranque**
JWT HS512 + BCrypt(12)	**firma rápida sin PKI; hash adaptativo contra fuerza bruta**

Decisión	Porqué
hideUserNotFoundExceptions	**no filtrar usuarios existentes (anti-enumeración)**
Context-path /api + mismo origen nginx	**cero CORS en producción; cookies y CSRF simplificados**
adb reverse para Android físico	**evita exponer el backend a la LAN y configurar IPs**
Minimización Ley 1581	**solo la URL sale del dispositivo; cuerpos jamás**
Code-first (impeccable)	**equipo pequeño, iterar rápido sobre UI existente**

11. Incidentes reales diagnosticados y resueltos (nov-2026)

1. Backend no arrancaba: localhost:5432 hardcodeado -> Flyway sin conexión en cascada hasta Tomcat. Fix: SPRING_DATASOURCE_URL por entorno.
2. Password de Postgres: volumen creado con clave vieja -> down + recrear volumen.
3. Hash admin truncado (52 chars) -> login admin imposible. Fix: V8 + corrección en BD.
4. 401 fantasma en login/registro: token obsoleto en localStorage (firma válida, usuario inexistente) rompía hasta endpoints públicos. Fix: filtro tolerante + limpieza en api.ts.
5. Nginx duplicaba /api/api/... por proxy_pass con variable. Fix: proxy_pass http://$backend_host$request_uri.
6. Falso positivo XSS: sessionExpired= contenía onExpired=. Fix: lookbehind.
7. Android en segundo plano muerto: consentimiento sin guardar + permisos de sistema denegados (listener, SMS, notificaciones) + batería. Fix: guardar consentimiento y arrancar guardia, pedir permiso al probar, guía de activación.
8. UI Android con emojis: vectores + RiskUi (puntos semánticos), search clicable, toolbar con Up.

12. Deuda técnica y trabajo futuro

- Extensión y APK play sin pruebas E2E (verificar contra backend actual).
- Tests automatizados escasos (hay vitest.config en web; backend -DskipTests).
- dist/ y APKs compilados no deben versionarse (añadir a .gitignore).
- Rate limiting en memoria (migrar a Redis/Bucket4J en producción).
- SECRET/keys en .env plano (usar secretos de Docker o vault en prod).
- Accesibilidad WCAG sin auditar formalmente.

Anexos

A. Variables de entorno: DB_USERNAME, DB_PASSWORD, JWT_SECRET, GOOGLE_SAFE_BROWSING_KEY, VIRUSTOTAL_KEY, SPRING_DATASOURCE_URL (solo Docker), APP_SECURITY_TRUSTED_PROXIES, CONTENT_ANALYSIS_ENABLED. **B. Migraciones:** V1 esquema · V2 seed · V3 OWASP/NIST · V4 comunidad · V5 quiz · V6 fix quiz · V7 microlecciones · V8 fix admin. **C. Seed:** admin / Admin@phish2024 (roles ADMIN+USER, BCrypt cost 12 válido). **D. Puertos:** 3000 web (80 interno), 8080 api, 5432 postgres.